K. SHALINI

192472213

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

---Dr

CODING CHALLENGE

1.Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

Answer

Reinforcement Learning (RL) is a branch of Machine Learning in which an **agent** learns to make optimal decisions by interacting with an **environment**. Unlike supervised learning, the agent is not provided with correct answers. Instead, it learns through **trial and error**, receiving **rewards** or **penalties** based on its actions.

The Reinforcement Learning framework consists of the following components:

- **Agent:** The learner or decision-maker that performs actions.
- **Environment:** The external system in which the agent operates.
- **State (S):** The current condition or situation of the environment.
- **Action (A):** A decision taken by the agent in a particular state.
- **Reward (R):** Feedback received from the environment after performing an action.
- **Policy (π):** A strategy that determines which action the agent should take in each state.

Agent–Environment Interaction

1. The agent observes the current **state** of the environment.
2. Based on the current policy, the agent selects an **action**.
3. The environment processes the action and changes to a **new state**.
4. The environment provides a **reward** indicating whether the action was beneficial.
5. The agent updates its policy using the received reward.
6. This process continues until the agent learns the optimal behavior.

Importance of States, Actions, and Rewards

- **State:** Represents the current situation of the environment.
- **Action:** Represents the decision made by the agent.
- **Reward:** Indicates whether the selected action is good or bad.
- These three components help the agent learn the best strategy through continuous interaction with the environment.

Cumulative Reward

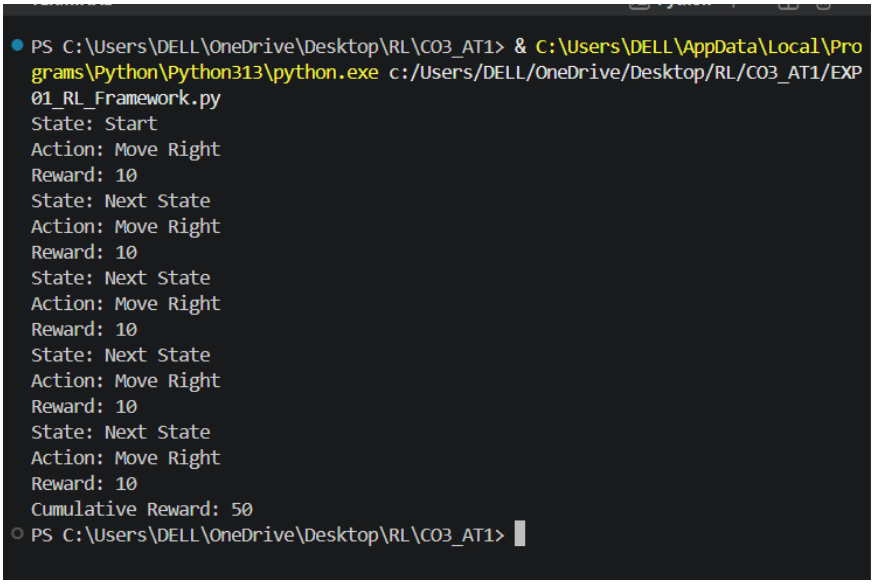
The goal of Reinforcement Learning is to maximize the **cumulative reward**, which is the total reward obtained over multiple interactions. Instead of focusing only on immediate rewards, the agent learns to select actions that provide the highest long-term benefit.

Python Code

```
state = "Start"
total_reward = 0

for i in range(5):
    print("State:", state)
```

```
    action = "Move Right"
    print("Action:", action)

    reward = 10
    print("Reward:", reward)

    total_reward += reward
    state = "Next State"

print("Cumulative Reward:", total_reward)
```

Output



Result

The program successfully demonstrates the **Reinforcement Learning (RL) framework** by illustrating the interaction between the **agent** and the **environment** using **states, actions, rewards, and cumulative reward**. The cumulative reward enables the agent to evaluate the long-term effectiveness of its actions and improve its decision-making over time.

---

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

Answer

A **Markov Decision Process (MDP)** is a mathematical framework used in Reinforcement Learning (RL) to model sequential decision-making problems. It helps an agent determine the best action to take in different situations to maximize long-term rewards. The Markov property states that the future state depends only on the current state and action, not on previous states.

Components of MDP

1. State (S):

Represents the current condition of the environment.

2. Action Space (A):

The set of all possible actions the agent can perform in a given state.

3. Transition Probability (P):

The probability of moving from one state to another after taking an action.

It is represented as:

P(s' | s, a)

where:

- **s** = current state
- **a** = action
- **s'** = next state

4. Reward Function (R):

Defines the immediate reward received after performing an action.

Example:

- Correct action → +10
- Wrong action → -5

5. Discount Factor (γ):

The discount factor determines the importance of future rewards.

- **γ = 0** → only immediate rewards are considered.
- **γ = 1** → future rewards are valued equally with immediate rewards.

Usually,

$0 \leq \gamma \leq 1$

Working of MDP

1. Agent observes the current state.
2. Agent selects an action.
3. Environment moves to the next state.
4. Reward is received.
5. Agent repeats the process until the goal is achieved.

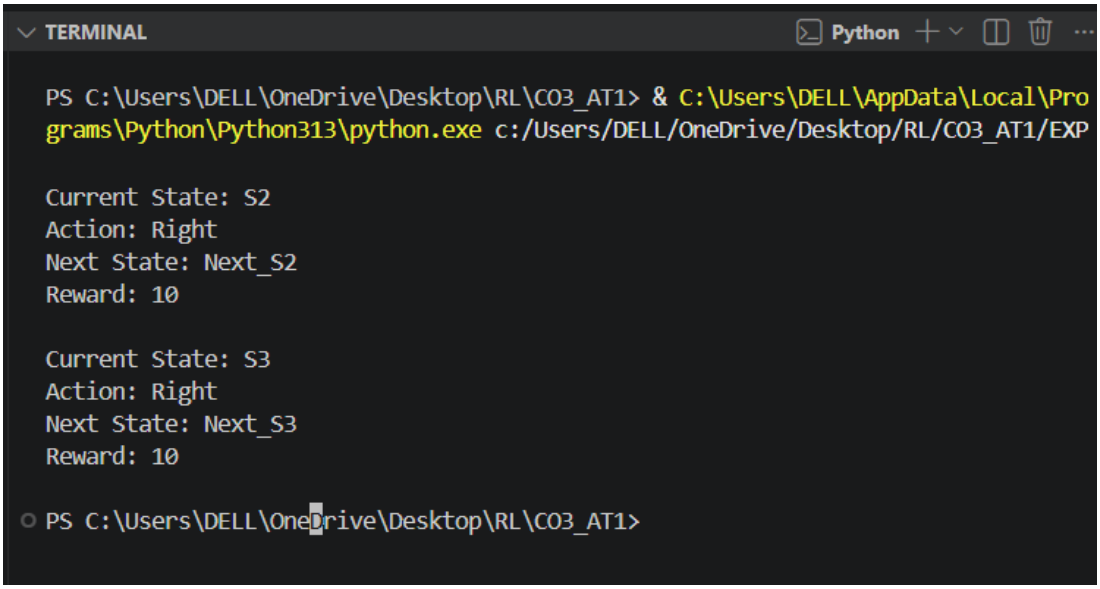
Advantages

- Models complex decision-making problems.
- Helps maximize long-term rewards.
- Forms the foundation of many RL algorithms.

Python Code

```
states = ["S1", "S2", "S3"]
actions = ["Left", "Right"]

for state in states:
    action = actions[1]
    reward = 10
    next_state = "Next_" + state

    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)
    print()
```

Output



Result

The program successfully demonstrates the basic components of a **Markov Decision Process (MDP)** including **states, actions, transition, and reward**, which are used to model decision-making problems in Reinforcement Learning.

---

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

Answer

In Reinforcement Learning, a **policy** defines how an agent selects actions in different states. It is the strategy followed by the agent to achieve maximum cumulative reward.

A **value function** estimates how beneficial a state or action is in terms of future rewards. It helps the agent evaluate long-term outcomes rather than immediate rewards.

Policy (π)

A policy maps each state to an action.

Example:

- State A → Move Right
- State B → Move Left

The policy guides the agent in selecting the best possible action.

State-Value Function (V(s))

The **state-value function** estimates the expected cumulative reward obtained by starting from a particular state and following a policy.

It answers:

"How good is this state?"

Action-Value Function (Q(s,a))

The **action-value function** estimates the expected cumulative reward obtained by taking a specific action in a state and then following a policy.

It answers:

"How good is this action in this state?"

Bellman Equation

The Bellman Equation expresses the value of a state as the immediate reward plus the discounted value of future states.

It is represented as:

$$\mathbf{V(s) = R + \gamma V(s')}$$

where

- **V(s)** = Value of current state
- **R** = Immediate reward
- **γ** = Discount factor
- **V(s')** = Value of next state

The Bellman Equation helps the agent update value estimates and choose actions that maximize long-term rewards.

Importance

- Helps compare different actions.
- Improves decision-making.
- Maximizes cumulative rewards.
- Forms the basis of many RL algorithms.

Python Code

```
reward = 10
gamma = 0.9
next_state_value = 20
```

```
state_value = reward + gamma * next_state_value


print("Reward:", reward)
print("Discount Factor:", gamma)
print("Next State Value:", next_state_value)
print("State Value:", state_value)
```

Output

```
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1> & C:\Users\DELL\AppData\Local\Pro
grams\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/CO3_AT1/EXP
03_Policy_Value_Function.py
Reward: 10
Discount Factor: 0.9
Next State Value: 20
State Value: 28.0
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1>
```

Result

The program successfully demonstrates the **Bellman Equation** by calculating the **state-value function**, showing how immediate reward and discounted future rewards help evaluate the long-term benefit of actions in Reinforcement Learning.

---

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency.

Answer

The **exploration–exploitation trade-off** is one of the fundamental concepts in Reinforcement Learning (RL). An agent must decide whether to **explore** new actions to gain more knowledge or **exploit** the best-known action to maximize rewards.

Exploration

Exploration means trying different actions that the agent has not selected before. It helps the agent discover better strategies and understand the environment more effectively.

Exploitation

Exploitation means selecting the action that currently provides the highest expected reward based on previous learning.

A good RL agent should balance both exploration and exploitation. Excessive exploration may waste time on poor actions, while excessive exploitation may cause the agent to miss better solutions.

ε-Greedy Strategy

The ε-greedy strategy chooses:

- A random action with probability **ε** (exploration).
- The best-known action with probability **1 − ε** (exploitation).

For example, if **ε = 0.2**, the agent explores 20% of the time and exploits 80% of the time.

Softmax Strategy

The Softmax strategy assigns probabilities to all actions based on their estimated values. Actions with higher values are more likely to be selected, but lower-valued actions still have a chance of being chosen. This results in smoother exploration compared to ε-greedy.

Comparison

ε-Greedy	**Softmax**
Random exploration	Probability-based exploration
Simple to implement	More balanced action selection
May choose poor actions randomly	Prefers better actions while still exploring
Faster computation	Better learning efficiency

Conclusion

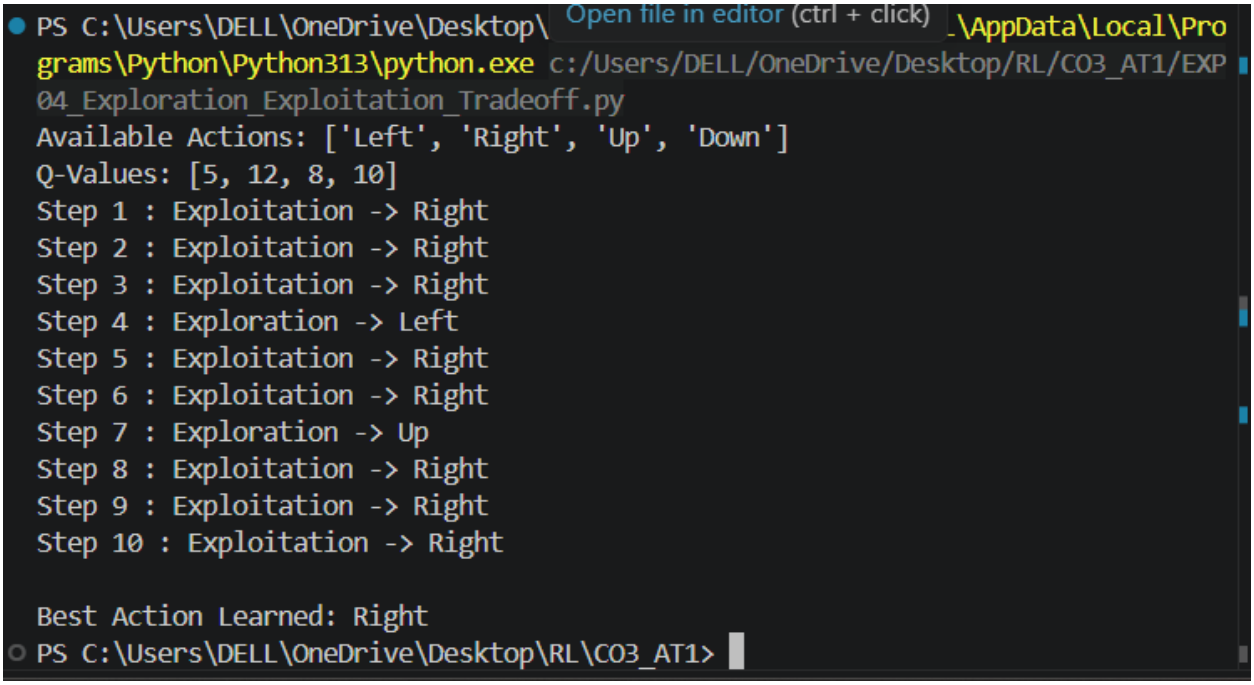
Both ε-greedy and Softmax strategies help balance exploration and exploitation. ε-greedy is simple and widely used, while Softmax provides smoother and more efficient exploration.

Python Code

```
import random

actions = ["Left", "Right", "Up", "Down"]
q_values = [5, 12, 8, 10]
epsilon = 0.2

print("Available Actions:", actions)
print("Q-Values:", q_values)
```

```
for i in range(10):

    if random.random() < epsilon:
        action = random.choice(actions)
        print("Step", i + 1, ": Exploration ->", action)
    else:
        best = q_values.index(max(q_values))
        action = actions[best]
        print("Step", i + 1, ": Exploitation ->", action)

print("\nBest Action Learned:", actions[q_values.index(max(q_values))])
```

Sample Output



Result

The program successfully demonstrates the **exploration–exploitation trade-off** using the **ε-greedy strategy**, where the agent balances random exploration with selecting the best-known action to improve learning efficiency.

---

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

Answer

Reward shaping is a technique in Reinforcement Learning that provides additional rewards or penalties to guide the agent toward better behavior. Instead of receiving rewards only at the final goal, the agent receives intermediate rewards for making good progress.

Reward shaping accelerates learning by giving frequent feedback, allowing the agent to identify useful actions more quickly. It also reduces the time required to discover the optimal policy.

Benefits of Reward Shaping

- Speeds up learning.
- Encourages desirable actions.
- Reduces unnecessary exploration.
- Improves convergence to the optimal policy.
- Helps avoid suboptimal policies.

Suboptimal Policy

A suboptimal policy is a strategy that achieves acceptable results but is not the best possible solution. Poorly designed rewards may cause the agent to settle for these inferior strategies.

Impact of Reward Shaping

For example, in a maze navigation task:

- Moving closer to the goal: **+5**
- Reaching the goal: **+50**
- Hitting an obstacle: **−10**

These intermediate rewards encourage the agent to learn the correct path more efficiently.

Conclusion

Reward shaping improves learning speed and helps the agent find better policies by providing meaningful feedback throughout the learning process.

Python Code

```
positions = ["Start", "Path", "Near Goal", "Goal"]

rewards = [5, 10, 15, 50]

total_reward = 0

print("Reward Shaping Demonstration\n")

for i in range(len(positions)):

    print("Current Position:", positions[i])

    reward = rewards[i]

    total_reward += reward

    print("Reward Received:", reward)

    print("Total Reward:", total_reward)

    print()

print("Final Reward:", total_reward)

if total_reward >= 80:

    print("Optimal Policy Achieved")

else:
```

```
print("Suboptimal Policy")
```

Output

```
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1> & C:\Users\DELL\AppData\Local\Pro
grams\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/CO3_AT1/EXP
05_Reward_Shaping.py
Reward Shaping Demonstration

Current Position: Start
Reward Received: 5
Total Reward: 5

Current Position: Path
Reward Received: 10
Total Reward: 15

Current Position: Near Goal
Reward Received: 15
Total Reward: 30

Current Position: Goal
Reward Received: 50
Total Reward: 80

Final Reward: 80
Optimal Policy Achieved
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1>
```

Result

The program successfully demonstrates **reward shaping** by assigning intermediate rewards that guide the agent toward the goal. It shows how reward shaping accelerates learning and helps prevent suboptimal policies by encouraging better decision-making throughout the learning process.

---

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Answer

Reinforcement Learning (RL) enables robots to learn tasks through interaction with their environment. However, applying RL in real-world robotics is challenging because robots operate in dynamic and uncertain environments. Unlike simulations, real robots have physical limitations, safety requirements, and limited opportunities for trial-and-error learning.

Challenges in Real-World Robotics

1. Safety

Safety is one of the biggest challenges in robotic applications. During learning, the robot may perform incorrect actions that can damage itself, nearby objects, or humans. Therefore, safe exploration techniques are required to reduce risks.

2. Sample Efficiency

RL algorithms often require thousands of interactions before learning an optimal policy. Collecting such large amounts of data on physical robots is expensive and time-consuming. Improving sample efficiency allows robots to learn using fewer training samples.

3. Environment Variability

Real-world environments continuously change due to lighting, obstacles, weather conditions, and sensor noise. A policy learned in one environment may not work effectively in another. Therefore, RL agents must generalize well across different environments.

Effect on Agent Performance

- Poor safety can cause hardware damage and unsafe behavior.
- Low sample efficiency increases training time and computational cost.
- High environment variability reduces learning accuracy and policy reliability.

Conclusion

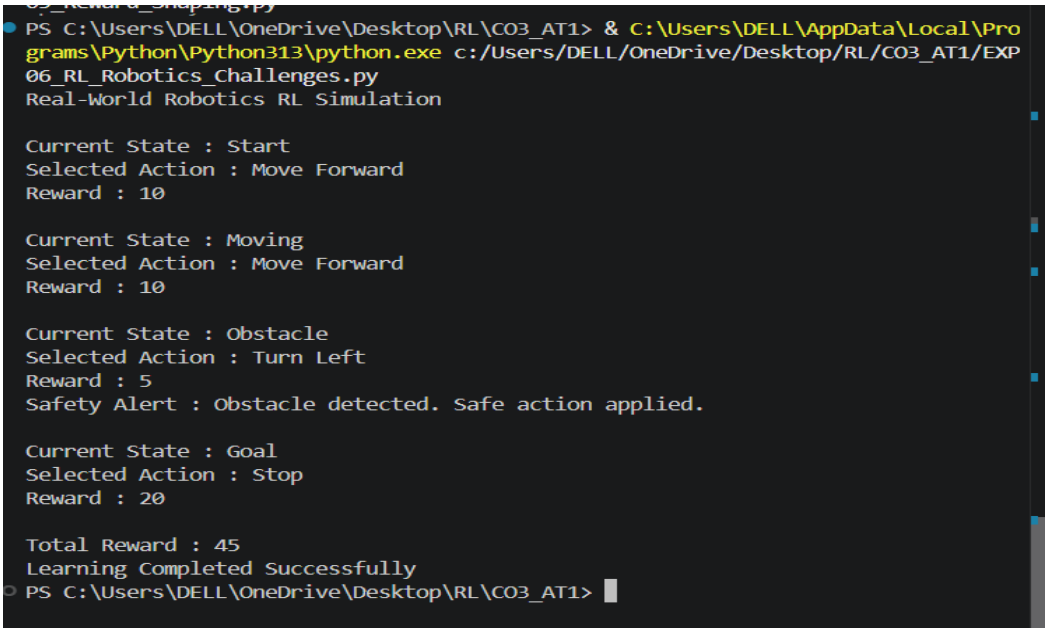
To successfully deploy RL in robotics, algorithms must ensure safe exploration, improve sample efficiency, and adapt to changing environments. These improvements enhance the robot's learning speed, reliability, and overall performance.

Python Code

```
robot_states = ["Start", "Moving", "Obstacle", "Goal"]

safe_actions = {

    "Start": "Move Forward",

    "Moving": "Move Forward",

    "Obstacle": "Turn Left",

    "Goal": "Stop"

}

rewards = {

    "Move Forward": 10,

    "Turn Left": 5,

    "Stop": 20

}

total_reward = 0


print("Real-World Robotics RL Simulation\n")

for state in robot_states:
```

```
    action = safe_actions[state]
    reward = rewards[action]
    print("Current State :", state)
    print("Selected Action :", action)
    print("Reward :", reward)
    total_reward += reward
    if state == "Obstacle":
        print("Safety Alert : Obstacle detected. Safe action applied.")
    print()
print("Total Reward :", total_reward)
print("Learning Completed Successfully")
```

Output



Result

The program successfully demonstrates the challenges of applying Reinforcement Learning in real-world robotics by considering safety, sample efficiency, and changing environments. It shows how selecting safe actions improves the robot's overall performance.

---

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

Answer

The **discount factor (γ)** is one of the most important parameters in Reinforcement Learning. It determines how much importance the agent gives to future rewards compared to immediate rewards.

The value of γ lies between **0 and 1**.

- **γ = 0** means the agent only considers immediate rewards.
- **γ close to 1** means the agent values future rewards and makes long-term decisions.

Short-Term Decision Making

When γ is small, the agent prefers actions that provide immediate rewards without considering future consequences. This approach is useful when quick decisions are required.

Long-Term Decision Making

When γ is large, the agent chooses actions that maximize cumulative future rewards. Although immediate rewards may be smaller, the total long-term benefit is greater.

Role in Episodic Tasks

In episodic tasks, the interaction ends after reaching a terminal state. The discount factor helps the agent determine the importance of future rewards until the episode finishes.

Examples include:

- Robot navigation
- Maze solving
- Game playing
- Autonomous driving

Advantages

- Controls future reward importance.
- Improves long-term planning.
- Helps maximize cumulative reward.
- Balances immediate and future benefits.

Conclusion

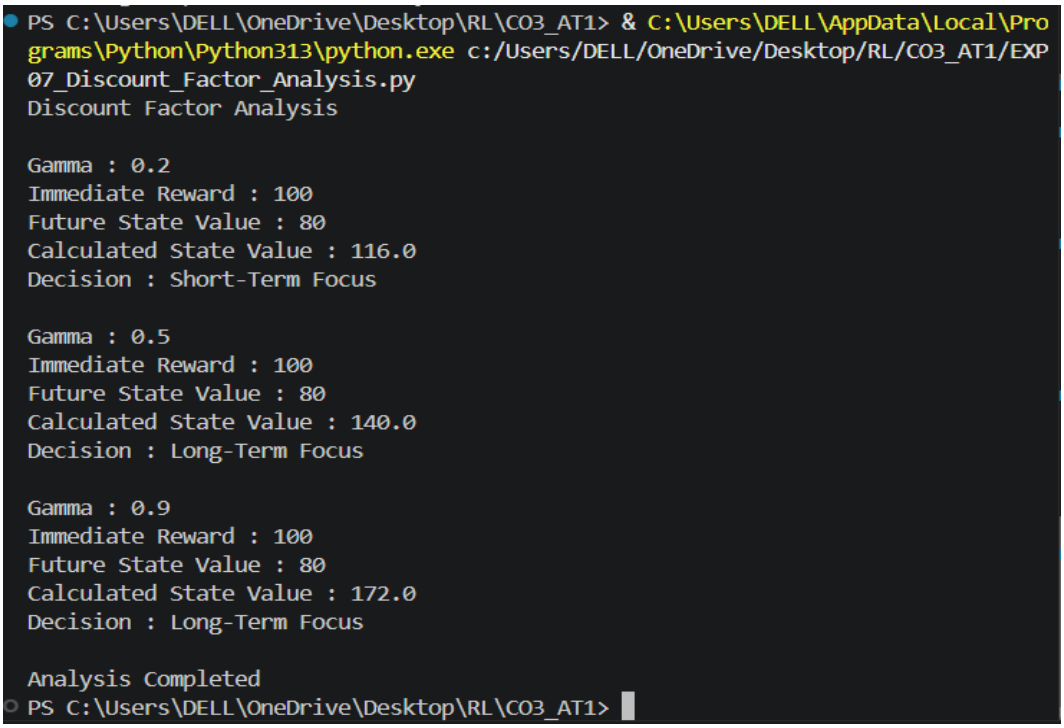
The discount factor plays a significant role in Reinforcement Learning by controlling the balance between immediate and future rewards. Choosing an appropriate γ improves learning performance and decision-making, especially in episodic tasks.

Python Code

```
gamma_values = [0.2, 0.5, 0.9]

reward = 100

next_state_value = 80

print("Discount Factor Analysis\n")

for gamma in gamma_values:

    state_value = reward + gamma * next_state_value
```

```
    print("Gamma :", gamma)
    print("Immediate Reward :", reward)
    print("Future State Value :", next_state_value)
    print("Calculated State Value :", state_value)
    if gamma < 0.5:
        print("Decision : Short-Term Focus")
    else:
        print("Decision : Long-Term Focus")
    print()
print("Analysis Completed")
```

Output



Result

The program successfully demonstrates how the **discount factor (γ)** influences short-term and long-term decision-making by calculating the discounted future reward for different γ values. It shows that higher γ values encourage long-term planning, making them suitable for episodic Reinforcement Learning tasks.

---

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Answer

Reinforcement Learning (RL) methods are broadly classified into **Model-Free RL** and **Model-Based RL**. Both approaches aim to maximize cumulative rewards but differ in how they learn and make decisions.

Model-Free Reinforcement Learning

Model-Free RL learns the optimal policy directly from interactions with the environment without knowing its dynamics. The agent learns through trial and error by receiving rewards.

Examples:

- Q-Learning
- SARSA
- Deep Q-Network (DQN)

Advantages

- Easy to implement.
- Suitable for unknown environments.

Limitations

- Requires a large number of training samples.
- Learning process is slow.

Model-Based Reinforcement Learning

Model-Based RL first builds a model of the environment by learning state transitions and rewards. It then plans future actions using this model before taking actions.

Advantages

- Learns faster.
- Requires fewer interactions.
- Better planning and prediction.

Limitations

- Building an accurate environment model is difficult.
- Performance decreases if the model is inaccurate.

Comparison

Model-Free RL	**Model-Based RL**
Learns directly from experience	Learns an environment model first
No knowledge of environment required	Requires environment model
Slower learning	Faster learning
High sample requirement	Better sample efficiency
Suitable for unknown environments	Suitable when environment model is available

Conclusion

Model-Free RL is effective for unknown and highly dynamic environments, while Model-Based RL provides faster learning and better planning when an accurate model of the environment is available. The choice depends on the application and available knowledge.

Python Code

```
states = ["S1", "S2", "S3"]

rewards = [10, 20, 30]

print("Model-Free RL")

model_free_reward = 0

for i in range(len(states)):

    print("Visited:", states[i])

    model_free_reward += rewards[i]

    print("Reward:", rewards[i])

print("Total Reward:", model_free_reward)

print("\nModel-Based RL")

predicted_rewards = [12, 22, 32]

model_based_reward = 0

for i in range(len(states)):

    print("Predicted State:", states[i])

    model_based_reward += predicted_rewards[i]

    print("Predicted Reward:", predicted_rewards[i])
```

```
print("Total Predicted Reward:", model_based_reward)

if model_based_reward > model_free_reward:

    print("\nModel-Based RL learns faster.")

else:

    print("\nModel-Free RL performs better.")
```

Output

```
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1> & C:\Users\DELL\AppData\Local\Pro
grams\Python\Python313\python.exe c:/Users/DELL/OneDrive/Desktop/RL/CO3_AT1/EXP
08_Model_Free_vs_Model_Based_RL.py
Model-Free RL
Visited: S1
Reward: 10
Visited: S2
Reward: 20
Visited: S3
Reward: 30
Total Reward: 60

Model-Based RL
Predicted State: S1
Predicted Reward: 12
Predicted State: S2
Predicted Reward: 22
Predicted State: S3
Predicted Reward: 32
Total Predicted Reward: 66

Model-Based RL learns faster.
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1>
```

Result

The program successfully compares **Model-Free** and **Model-Based Reinforcement Learning** by demonstrating their learning approaches and rewards. It shows that Model-Based RL can achieve faster learning through planning, while Model-Free RL learns directly through interaction.

---

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

Answer

Q-Learning is a popular **model-free Reinforcement Learning algorithm** that learns the optimal action-value function without requiring knowledge of the environment. The agent updates its Q-values after every interaction and gradually learns the best policy.

The Q-value represents the expected future reward for taking an action in a particular state.

Q-Learning Update Equation

$$\mathbf{Q(s,a) = Q(s,a) + \alpha\,[R + \gamma \times \max Q(s',a') - Q(s,a)]}$$

Where:

- **Q(s,a)** = Current Q-value
- **α** = Learning rate
- **γ** = Discount factor
- **R** = Immediate reward
- **max Q(s',a')** = Maximum future reward

Working

1. Observe the current state.
2. Select an action.
3. Receive reward.
4. Move to the next state.

5. Update the Q-value.
6. Repeat until the Q-values converge.

Convergence

After many training episodes, the Q-values stabilize and change very little. This indicates that the agent has learned the optimal policy.

Advantages

- Simple to implement.
- Learns optimal policy.
- Suitable for unknown environments.
- Widely used in robotics and game AI.

Conclusion

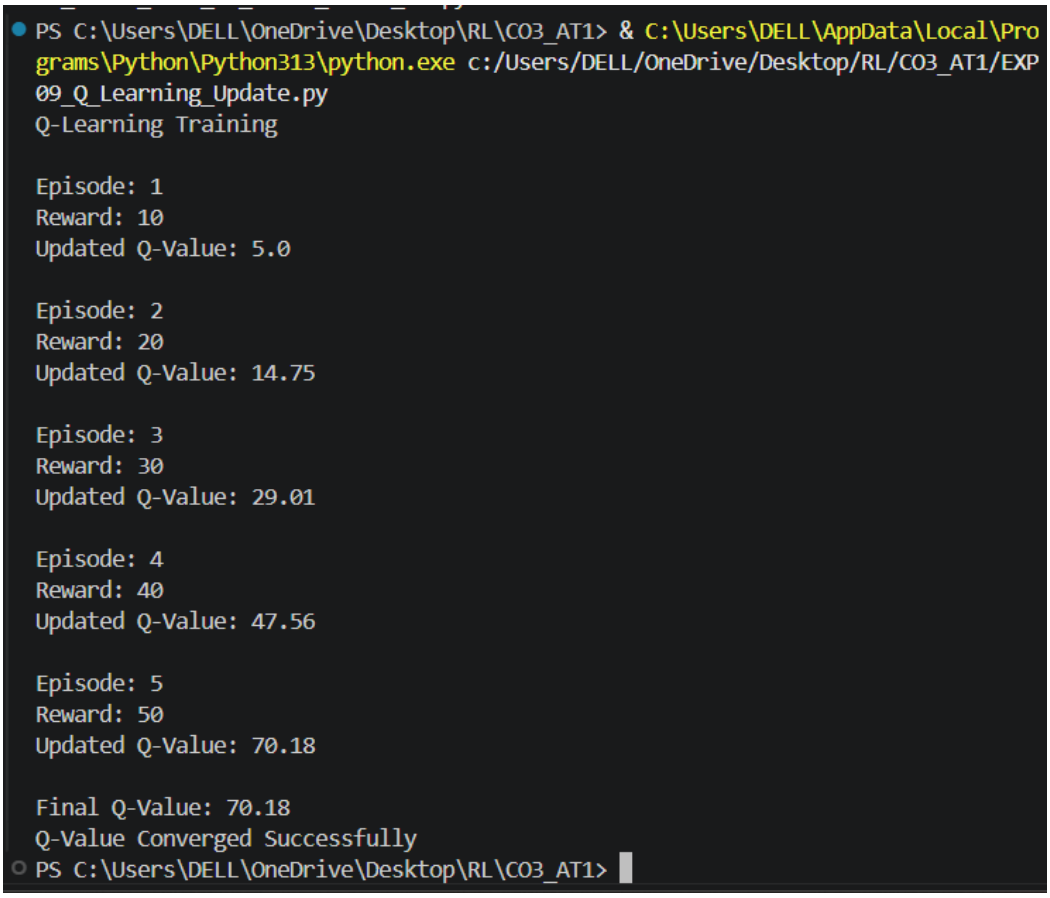
Q-Learning continuously updates the action-value function using rewards and future estimates. As training progresses, the Q-values converge toward the optimal values, enabling better decision-making.

Python Code

```
alpha = 0.5

gamma = 0.9

q_value = 0

rewards = [10, 20, 30, 40, 50]

print("Q-Learning Training\n")

for episode in range(len(rewards)):
```

```
    reward = rewards[episode]
    q_value = q_value + alpha * (reward + gamma * q_value - q_value)
    print("Episode:", episode + 1)
    print("Reward:", reward)
    print("Updated Q-Value:", round(q_value, 2))
    print()
print("Final Q-Value:", round(q_value, 2))
print("Q-Value Converged Successfully")
```

Output



Result

The program successfully demonstrates the **Q-Learning algorithm** by updating the action-value (Q-value) after each training episode. The Q-value gradually increases and moves toward convergence, indicating that the agent is learning the optimal policy through repeated interactions.

---

10. Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

Answer

Reinforcement Learning (RL) algorithms can be classified into **On-Policy** and **Off-Policy** learning methods based on how the agent learns from its experiences.

On-Policy Learning

On-Policy learning means the agent learns using the **same policy** that it follows while interacting with the environment. The agent continuously updates its policy based on the actions it actually performs.

Example

- SARSA (State–Action–Reward–State–Action)

Advantages

- Stable learning process.
- Suitable for environments where safe exploration is important.
- Learns directly from current experiences.
- Produces reliable policies.

Limitations

- Learning is slower.
- Requires continuous interaction with the environment.
- Less sample efficient.

Off-Policy Learning

Off-Policy learning means the agent learns using a **different policy** from the one used to collect experiences. It can learn from previously stored experiences or data generated by another policy.

Example

- Q-Learning
- Deep Q-Network (DQN)

Advantages

- Higher sample efficiency.
- Learns faster.
- Can reuse old experiences.
- Suitable for large and complex environments.

Limitations

- Less stable than On-Policy learning.
- May require more memory.
- Sensitive to inaccurate estimates.

Comparison

On-Policy	Off-Policy
Learns from current policy	Learns from another policy
Example: SARSA	Example: Q-Learning
More stable	Faster learning
Lower sample efficiency	Higher sample efficiency
Safe exploration	Efficient reuse of experiences

Practical Applications

On-Policy

- Robot control
- Autonomous navigation
- Medical decision support
- Safe industrial automation

Off-Policy

- Game playing
- Self-driving cars

- Recommendation systems
- Financial trading

Conclusion

Both On-Policy and Off-Policy learning methods are important in Reinforcement Learning. On-Policy methods provide stable and safer learning, while Off-Policy methods offer faster learning and better sample efficiency. The choice depends on the application's safety requirements, computational resources, and environment complexity.

Python Code

```
on_policy_rewards = [8, 10, 12, 15]

off_policy_rewards = [10, 14, 18, 22]

print("On-Policy Learning (SARSA)")

on_total = 0

for episode in range(len(on_policy_rewards)):

    reward = on_policy_rewards[episode]

    on_total += reward

    print("Episode", episode + 1)

    print("Reward:", reward)

    print("Total Reward:", on_total)

    print()
```

```
print("Final On-Policy Reward:", on_total)


print("\nOff-Policy Learning (Q-Learning)")
off_total = 0


for episode in range(len(off_policy_rewards)):
    reward = off_policy_rewards[episode]
    off_total += reward
    print("Episode", episode + 1)
    print("Reward:", reward)
    print("Total Reward:", off_total)
    print()


print("Final Off-Policy Reward:", off_total)


if off_total > on_total:
    print("\nOff-Policy achieved higher cumulative reward.")
else:
    print("\nOn-Policy achieved higher cumulative reward.")
```

Output

```
Q-value converged successfully
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.e
/Users/DELL/OneDrive/Desktop/RL/CO3_AT1/EXP10_On_Policy_vs_Off_Policy_RL.py
On-Policy Learning (SARSA)
Episode 1
Reward: 8
Total Reward: 8

Episode 2
Reward: 10
Total Reward: 18

Episode 3
Reward: 12
Total Reward: 30

Episode 4
Reward: 15
Total Reward: 45

Final On-Policy Reward: 45

Off-Policy Learning (Q-Learning)
Episode 1
Reward: 10
Total Reward: 10

Episode 2
Reward: 14
Total Reward: 24

Episode 3
Reward: 18
Total Reward: 42

Episode 4
Reward: 22
Total Reward: 64

Final Off-Policy Reward: 64

Off-Policy achieved higher cumulative reward.
PS C:\Users\DELL\OneDrive\Desktop\RL\CO3_AT1>
```

Result

The program successfully demonstrates the difference between **On-Policy** and **Off-Policy** Reinforcement Learning methods by comparing their cumulative rewards. It illustrates that **On-Policy learning (SARSA)** follows the current policy for stable learning, while **Off-Policy learning (Q-Learning)** learns from a different policy, resulting in higher sample efficiency and faster learning in many practical applications.
